

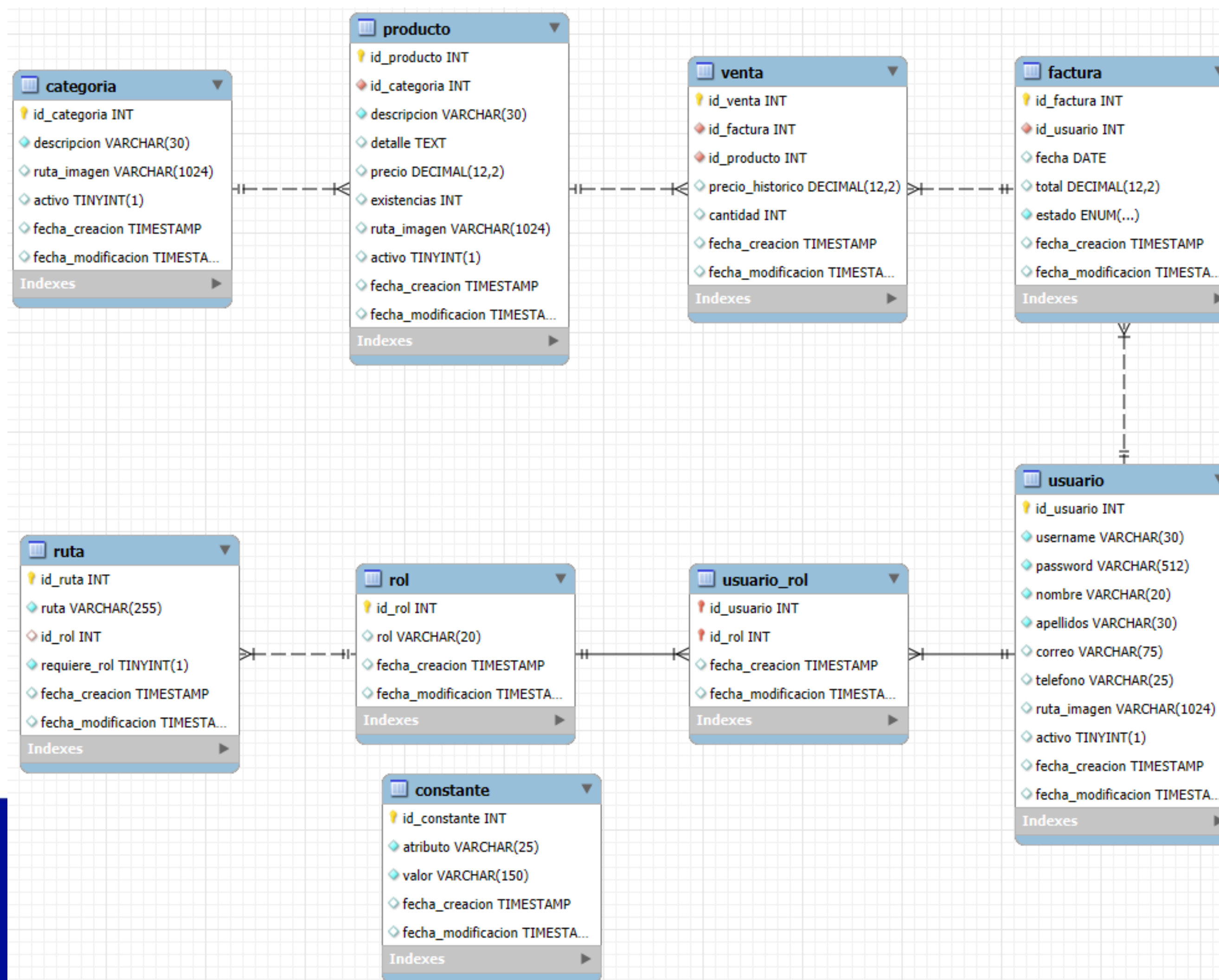
**Web y Patrones
Semana 6**

La interconexión de datos: Más allá de una sola tabla

Es raro que un sistema de información real se base en una única tabla para guardar todos sus datos. Piensa en una red social: no solo tienes usuarios, sino también publicaciones, comentarios, "me gusta" y mucho más. Si tratáramos de meter todo eso en una sola tabla, sería un caos.

Para que los datos sean coherentes, escalables y fáciles de gestionar, los sistemas de información están diseñados con múltiples tablas interconectadas, cada una con un propósito específico. Estas tablas no viven aisladas; tienen relaciones lógicas que reflejan el mundo real.

Esquema Entidad – Relación de la base de datos

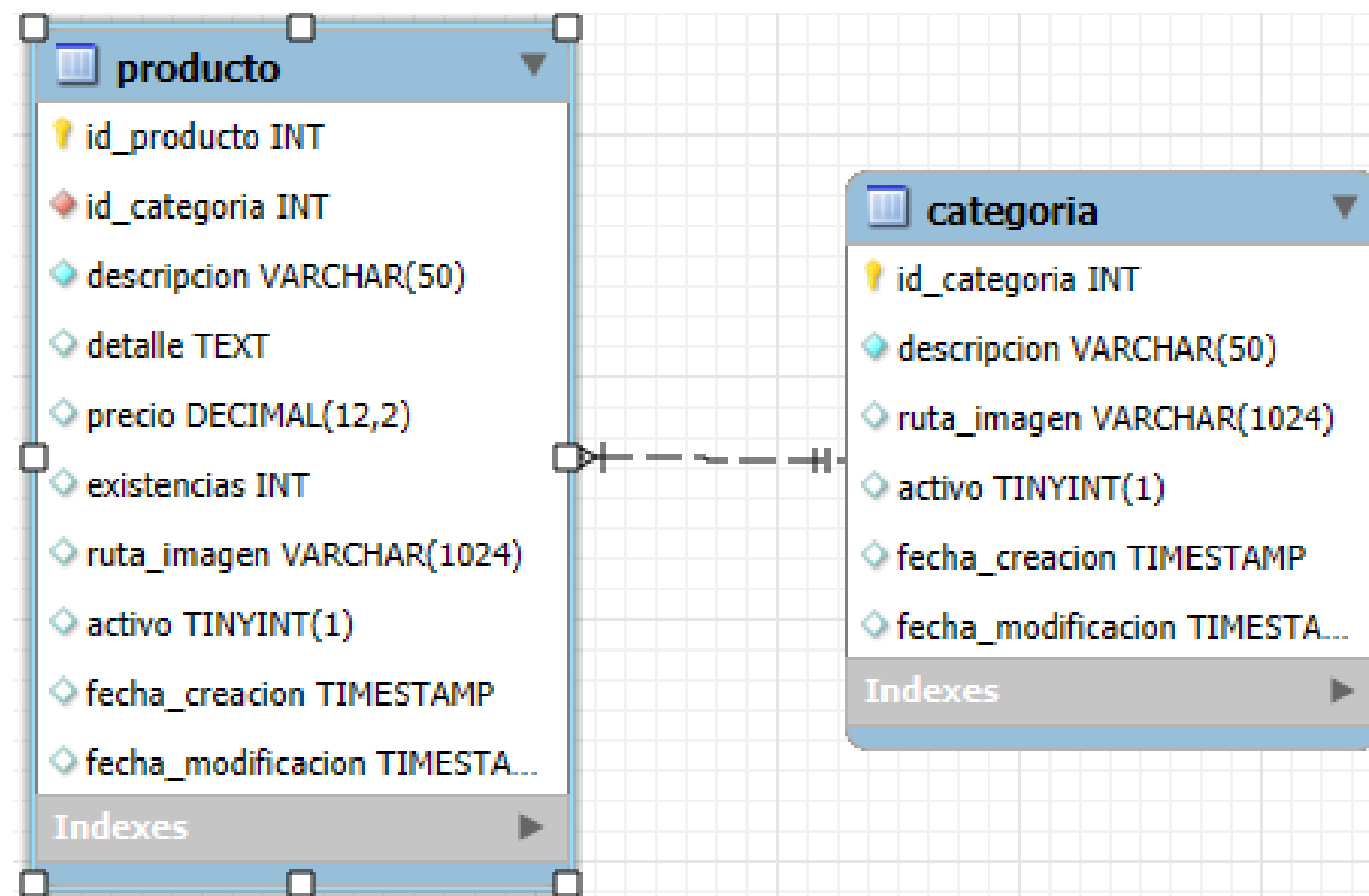


Asociación Muchos a Uno (producto -> categoría)

La anotación @ManyToOne en JPA Hibernate se utiliza para definir el lado "muchos" de una relación, donde muchas instancias de una entidad (como un Producto) pueden estar asociadas con una sola instancia de otra entidad (una Categoría).

En el esquema de la base de datos, la tabla producto tiene una columna id_categoria que es una clave foránea que apunta a la tabla categoría.

La anotación @ManyToOne es la forma de mapear esta columna en el código Java.



La nueva clase Producto

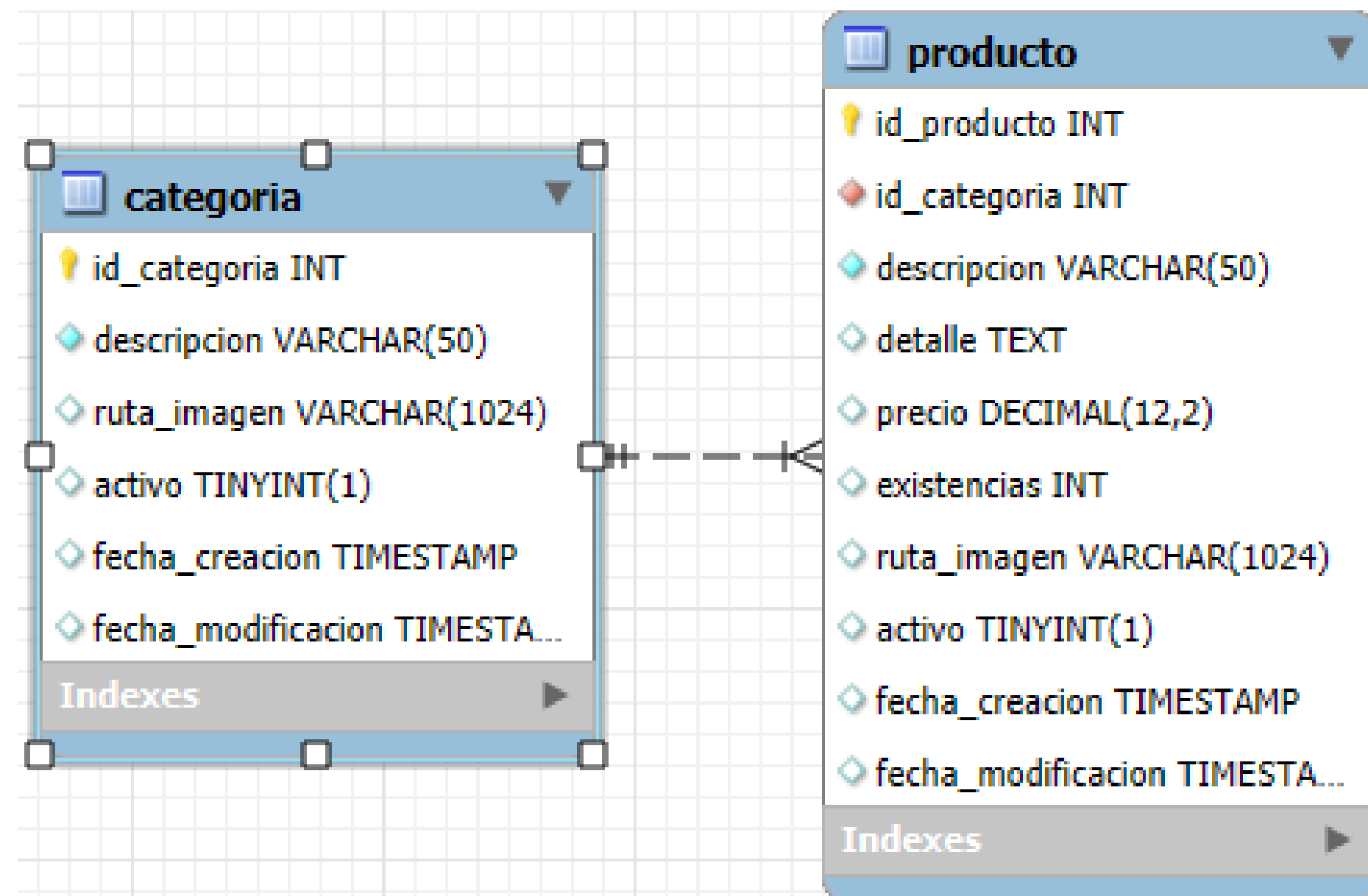
El atributo idCategoria no se utiliza, está implícito en el nuevo objeto categoria.

```
Producto.java X
Source History
9  @Data
10 @Entity
11 @Table(name = "producto")
12 public class Producto implements Serializable {
13
14     private static final long serialVersionUID = 1L;
15
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     @Column(name = "id_producto")
19     private Integer idProducto;
20     //private Integer idCategoria; ya no se usa por la anotación @ManyToOne
21
22     @Column(nullable = false, length = 50)
23     @NotBlank(message = "La descripción no puede estar vacía.")
24     @Size(max = 50, message = "La descripción no puede tener más de 50 caracteres.")
25     private String descripcion;
26
27     @Column(columnDefinition = "TEXT")
28     private String detalle;
29
30     @Column(precision = 12, scale = 2)
31     @NotNull(message = "El precio no puede estar vacío.")
32     @DecimalMin(value = "0.01", inclusive = true, message = "El precio debe ser mayor a 0.")
33     private BigDecimal precio;
34
35     @NotNull(message = "El campo de existencias no puede estar vacío.")
36     @Min(value = 0, message = "Las existencias deben ser un número mayor o igual a 0.")
37     private Integer existencias;
38
39     @Column(name = "ruta_imagen", length = 1024)
40     private String rutaImagen;
41     private boolean activo;
42
43     @ManyToOne
44     @JoinColumn(name = "id_categoria")
45     private Categoria categoria;
46 }
47
```

Asociación Uno a Muchos (categoria -> producto)

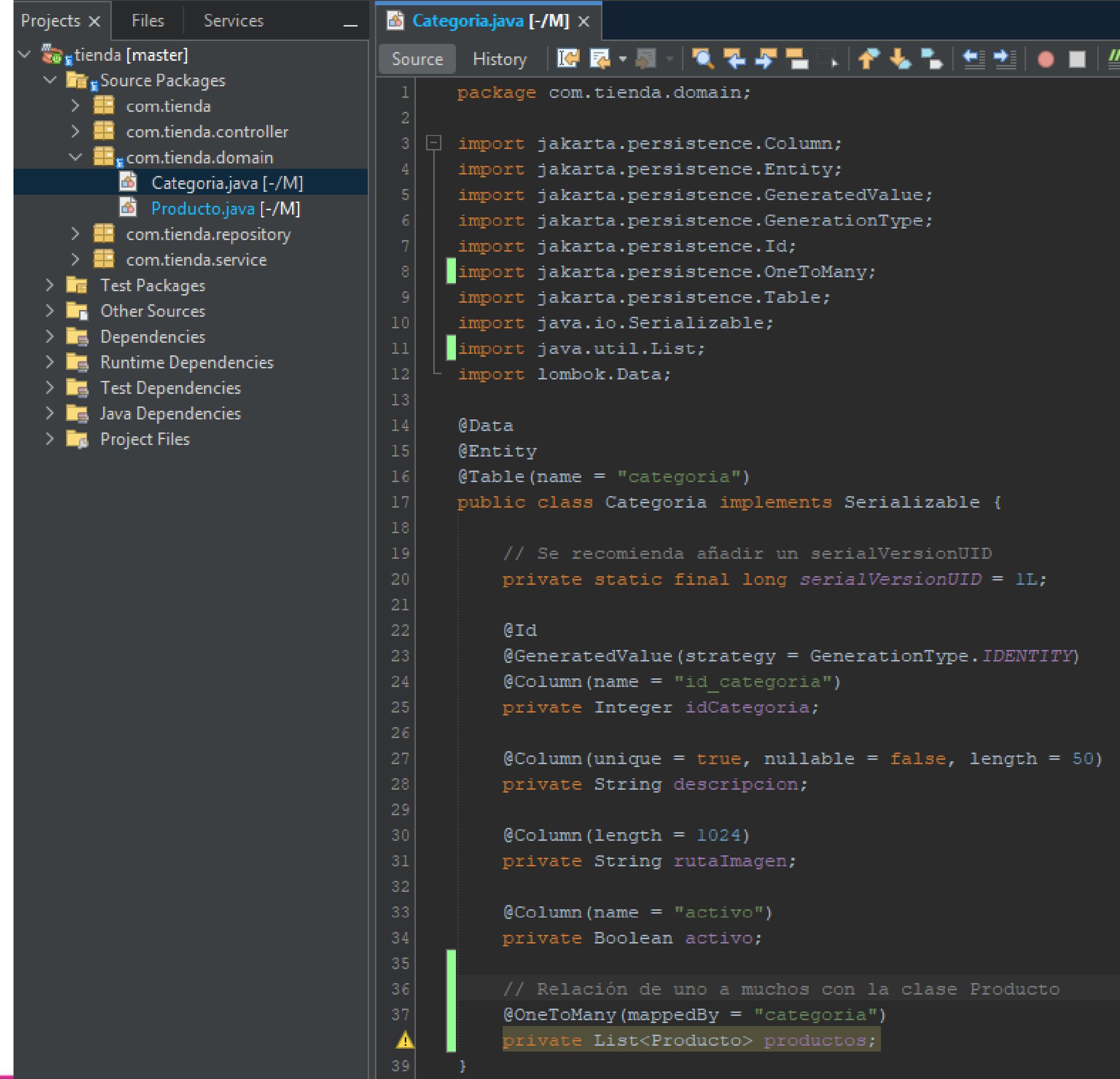
La anotación @OneToMany en JPA Hibernate se utiliza para definir una relación donde una entidad (la "uno", en este caso, Categoría) puede estar relacionada con múltiples instancias de otra entidad (la "muchos", en este caso, Producto).

Para mapear la relación "uno a muchos", se utiliza @OneToMany en la clase Categoría, ya que una categoría puede tener muchos productos. Este es el lado "propietario" de la relación, el que tiene la colección de objetos.



La nueva clase Categoria

El atributo idCategoria no se utiliza, está implícito en el nuevo objeto categoria.



```
1 package com.tienda.domain;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.GeneratedValue;
6 import jakarta.persistence.GenerationType;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.OneToMany;
9 import jakarta.persistence.Table;
10 import java.io.Serializable;
11 import java.util.List;
12 import lombok.Data;
13
14 @Data
15 @Entity
16 @Table(name = "categoria")
17 public class Categoria implements Serializable {
18
19     // Se recomienda añadir un serialVersionUID
20     private static final long serialVersionUID = 1L;
21
22     @Id
23     @GeneratedValue(strategy = GenerationType.IDENTITY)
24     @Column(name = "id_categoria")
25     private Integer idCategoria;
26
27     @Column(unique = true, nullable = false, length = 50)
28     private String descripcion;
29
30     @Column(length = 1024)
31     private String rutaImagen;
32
33     @Column(name = "activo")
34     private Boolean activo;
35
36     // Relación de uno a muchos con la clase Producto
37     @OneToMany(mappedBy = "categoria")
38     private List<Producto> productos;
39 }
```


Actualización de fragmentos

Se actualizan varias secciones de los fragmentos alrededor del uso de categoría, son 4 ajustes:

- Línea 34, en el name se cambia a name="categoría.idCategoría"
- Línea 109, igual ahora se coloca name="categoría.idCategoría"
- Línea 166, se hacen 2 ajustes

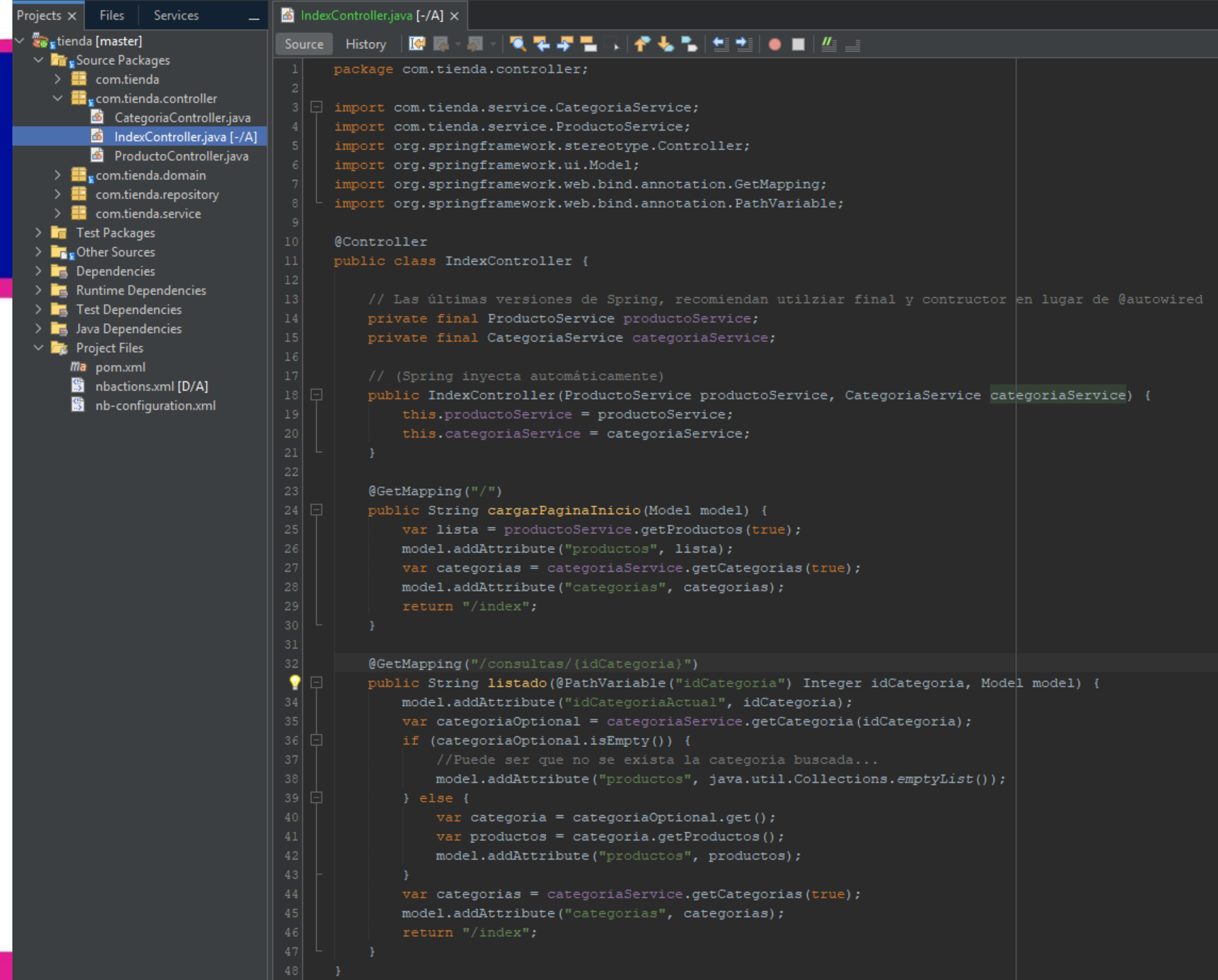
```
fragmentos.html [-/M] x
Source History
31 <div class="modal-body">
32 <div class="mb-3">
33 <label>[[#{producto.categoria}]]</label>
34 <select name="categoría.idCategoría"
35 class="form-select"
36 aria-label="Default select example">
```

```
fragmentos.html [-/M] x
Source History
109 <td>[[#{p.categoria.descripcion}]]</td>
110 <td th:text="${p.activo} ? 'Activa':'Inactiva'"></td>
```

```
fragmentos.html [-/M] x
Source History
165 <label>[[#{producto.categoria}]]</label>
166 <select name="categoría.idCategoría" th:field="*{categoría.idCategoría}"
167 class="form-select" aria-label="Default select example">
```


IndexController

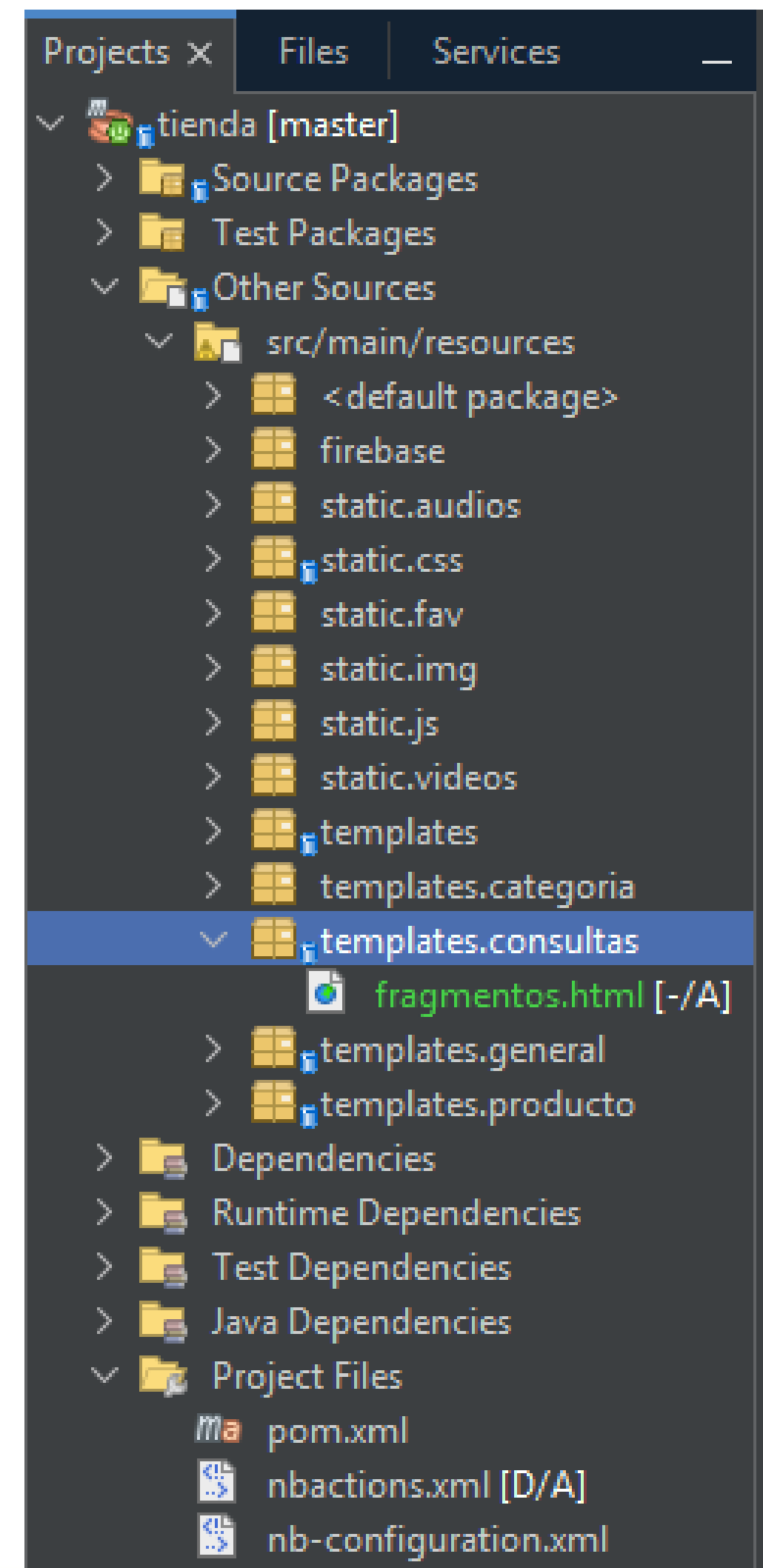
Ahora se crea un nuevo controlador llamado IndexController que se utilizará para esta consulta y otras que continúan en el curso. Se crea esta clase Controller a partir de ProductoController, dentro se actualiza casi todo.



```
1 package com.tienda.controller;
2
3 import com.tienda.service.CategoriaService;
4 import com.tienda.service.ProductoService;
5 import org.springframework.stereotype.Controller;
6 import org.springframework.ui.Model;
7 import org.springframework.web.bind.annotation.GetMapping;
8 import org.springframework.web.bind.annotation.PathVariable;
9
10 @Controller
11 public class IndexController {
12
13     // Las últimas versiones de Spring, recomiendan utilizar final y constructor en lugar de @Autowired
14     private final ProductoService productoService;
15     private final CategoriaService categoriaService;
16
17     // (Spring inyecta automáticamente)
18     public IndexController(ProductoService productoService, CategoriaService categoriaService) {
19         this.productoService = productoService;
20         this.categoriaService = categoriaService;
21     }
22
23     @GetMapping("/")
24     public String cargarPaginaInicio(Model model) {
25         var lista = productoService.getProductos(true);
26         model.addAttribute("productos", lista);
27         var categorias = categoriaService.getCategorias(true);
28         model.addAttribute("categorias", categorias);
29         return "/index";
30     }
31
32     @GetMapping("/consultas/{idCategoria}")
33     public String listado(@PathVariable("idCategoria") Integer idCategoria, Model model) {
34         model.addAttribute("idCategoriaActual", idCategoria);
35         var categoriaOptional = categoriaService.getCategoria(idCategoria);
36         if (categoriaOptional.isEmpty()) {
37             //Puede ser que no se exista la categoria buscada...
38             model.addAttribute("productos", java.util.Collections.emptyList());
39         } else {
40             var categoria = categoriaOptional.get();
41             var productos = categoria.getProductos();
42             model.addAttribute("productos", productos);
43         }
44         var categorias = categoriaService.getCategorias(true);
45         model.addAttribute("categorias", categorias);
46         return "/index";
47     }
48 }
```

Se crea el folder consultas

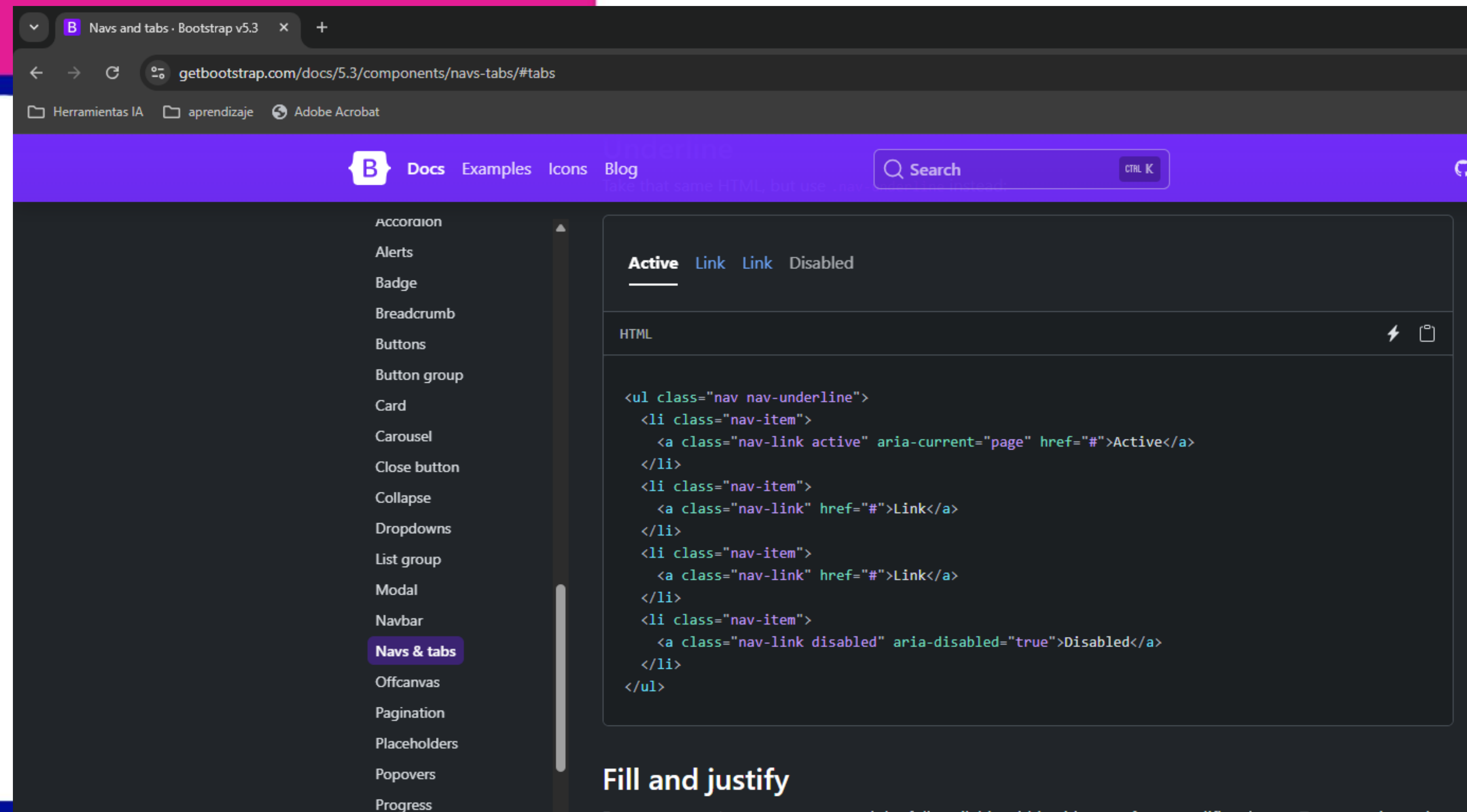
Se copia fragmentos de producto como fase inicial



fragmentos.html

Por esta semana se crean dos fragmentos
tabs y tarjetas.

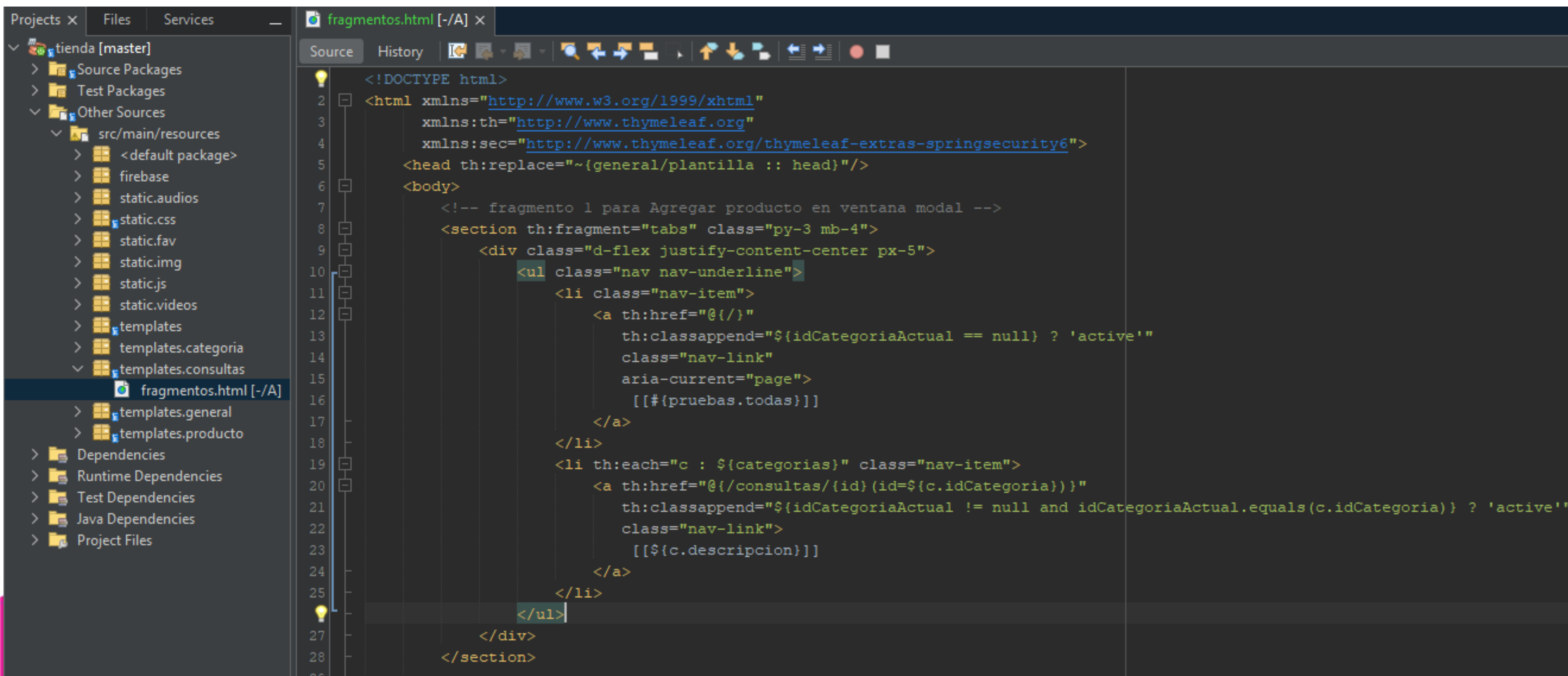
Es un excelente inicio, revisar “tabs” en
getbootstrap.com



El fragmento “tabs” en fragmentos.html

Con el se hará un enlace directo a los productos de cada categoría.

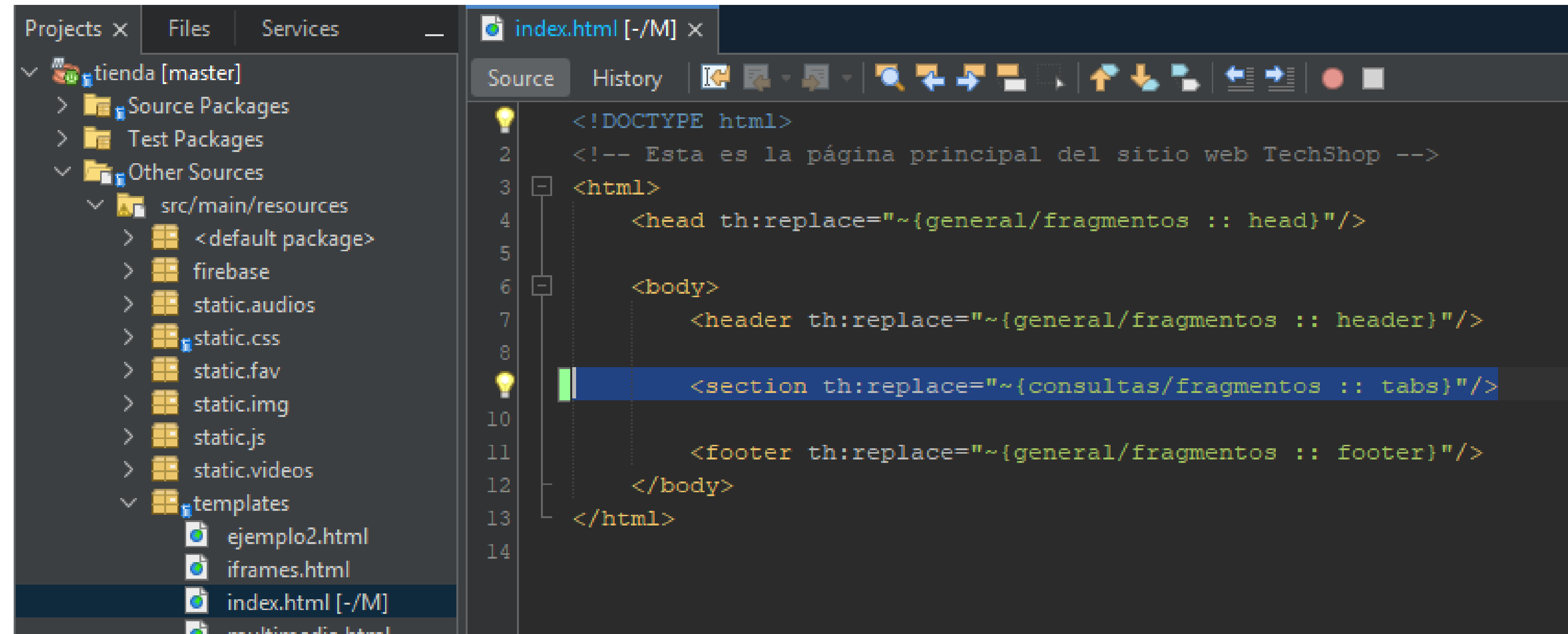
Se debe crear el enunciado del otro fragmento, si se omite da error la prueba.



```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org"
      xmlns:sec="http://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
<head th:replace="~{general/plantilla :: head}"/>
<body>
    <!-- fragmento 1 para Agregar producto en ventana modal -->
    <section th:fragment="tabs" class="py-3 mb-4">
        <div class="d-flex justify-content-center px-5">
            <ul class="nav nav-underline">
                <li class="nav-item">
                    <a th:href="@{/}"
                       th:classappend="${idCategoriaActual == null} ? 'active'"
                       class="nav-link"
                       aria-current="page">
                        [[#{pruebas.todas}]]
                    </a>
                </li>
                <li th:each="c : ${categorias}" class="nav-item">
                    <a th:href="@{/consultas/{id}(id=${c.idCategoria})}"
                       th:classappend="${idCategoriaActual != null and idCategoriaActual.equals(c.idCategoria)} ? 'active'"
                       class="nav-link">
                        [[#{c.descripcion}]]
                    </a>
                </li>
            </ul>
        </div>
    </section>
```

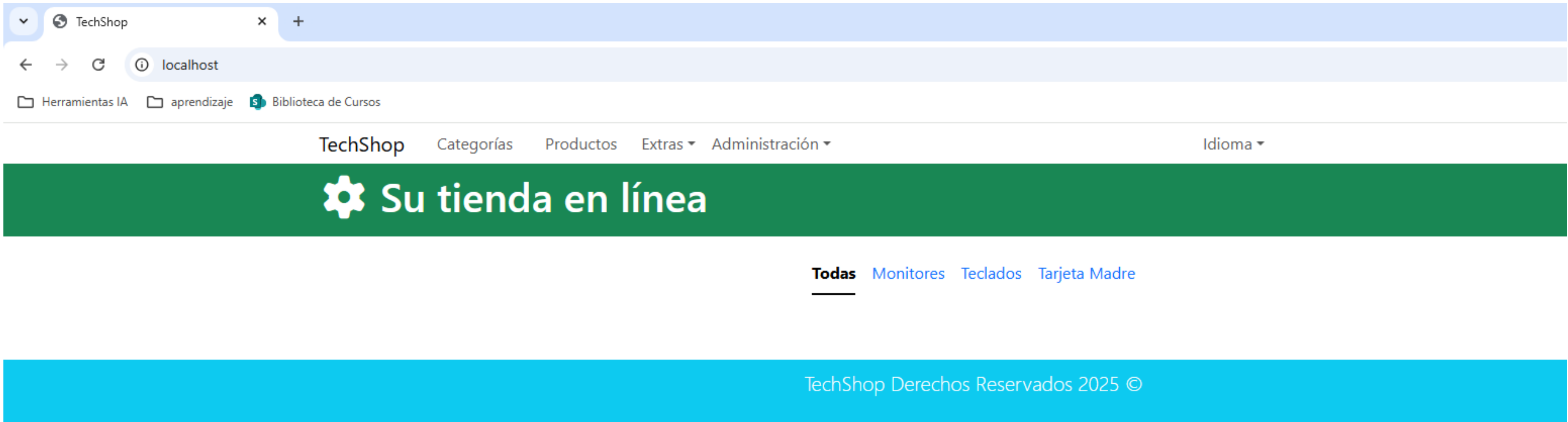
index.html

Se incluye el llamado al fragmento en el index.html

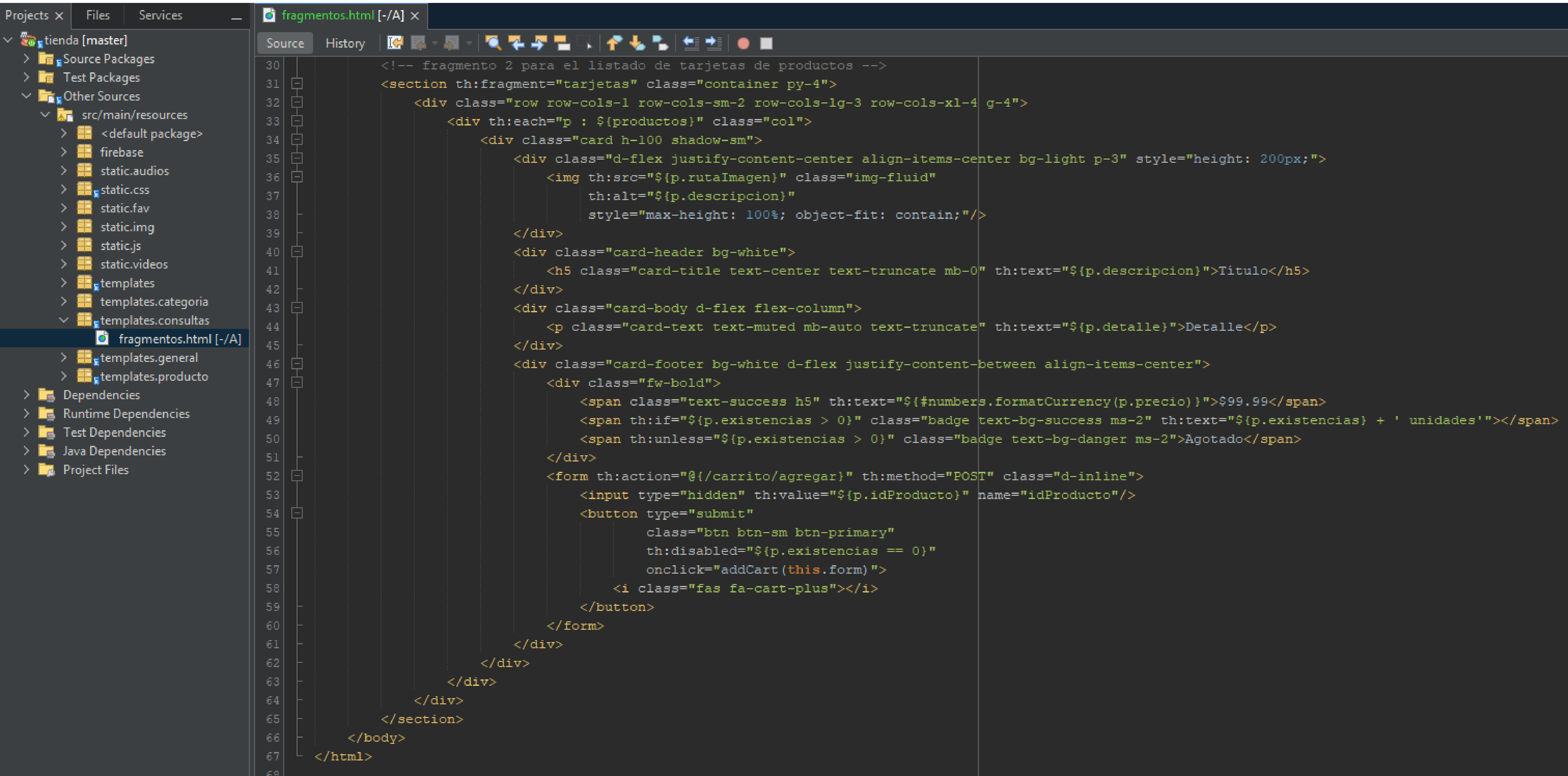


```
index.html [-/M] x
Source History
<!DOCTYPE html>
<!-- Esta es la página principal del sitio web TechShop -->
<html>
  <head th:replace="~{general/fragmentos :: head}"/>
  <body>
    <header th:replace="~{general/fragmentos :: header}"/>
    <section th:replace="~{consultas/fragmentos :: tabs}"/>
    <footer th:replace="~{general/fragmentos :: footer}"/>
  </body>
</html>
```

Así se observan las “tabs”



El fragmento “tarjetas” en fragmentos.html



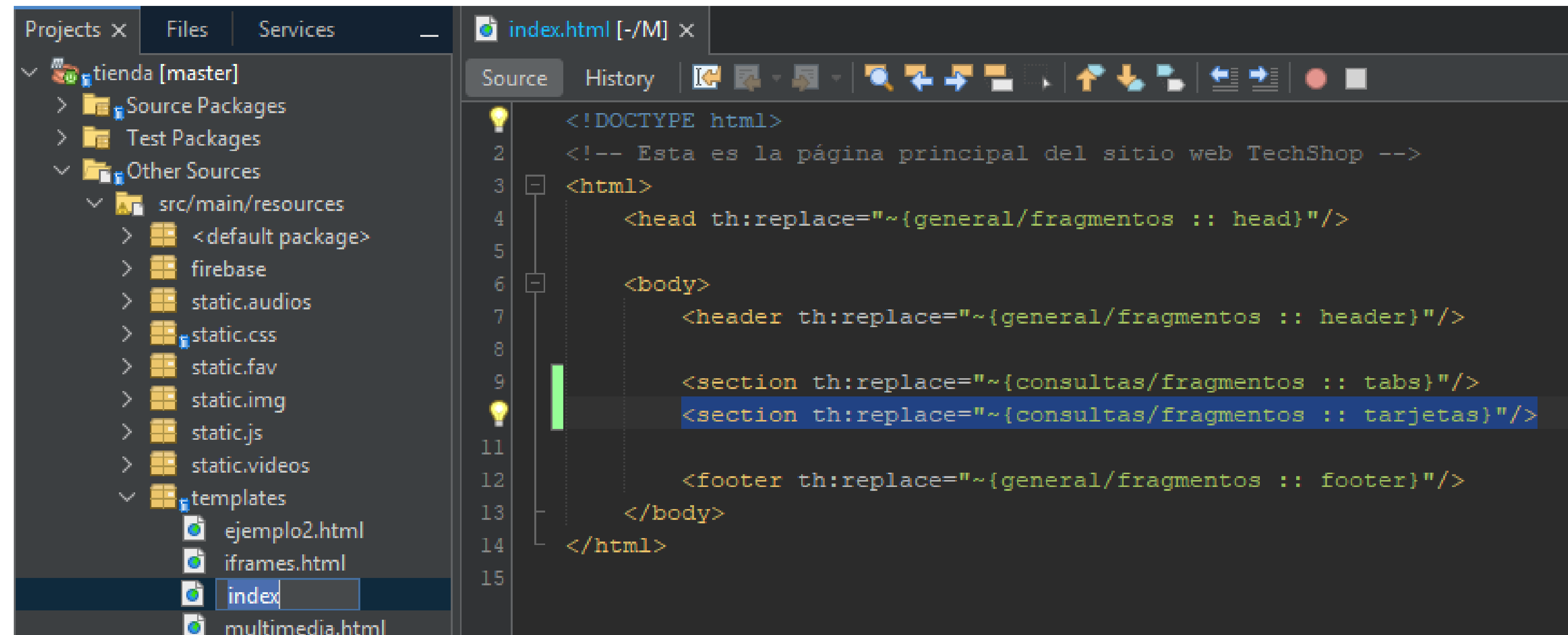
The image shows a screenshot of an IDE with a project named 'tienda [master]'. The left sidebar displays the project structure, including 'Source Packages', 'Test Packages', and 'Other Sources'. The 'Other Sources' folder contains a 'src/main/resources' directory with various static files and templates. The 'fragmentos.html' file is selected in the 'templates.consultas' subdirectory.

The main editor displays the code for the 'fragmentos.html' file, which is a Thymeleaf template. The code is structured as follows:

```
30 <!-- fragmento 2 para el listado de tarjetas de productos -->
31 <section th:fragment="tarjetas" class="container py-4">
32     <div class="row row-cols-1 row-cols-sm-2 row-cols-lg-3 row-cols-xl-4 g-4">
33         <div th:each="p : ${productos}" class="col">
34             <div class="card h-100 shadow-sm">
35                 <div class="d-flex justify-content-center align-items-center bg-light p-3" style="height: 200px;">
36                     
39                 </div>
40                 <div class="card-header bg-white">
41                     <h5 class="card-title text-center text-truncate mb-0" th:text="${p.descripcion}">Titulo</h5>
42                 </div>
43                 <div class="card-body d-flex flex-column">
44                     <p class="card-text text-muted mb-auto text-truncate" th:text="${p.detalle}">Detalle</p>
45                 </div>
46                 <div class="card-footer bg-white d-flex justify-content-between align-items-center">
47                     <div class="fw-bold">
48                         <span class="text-success h5" th:text="${#numbers.formatCurrency(p.precio)}">$99.99</span>
49                         <span th:if="${p.existencias > 0}" class="badge text-bg-success ms-2" th:text="${p.existencias} + ' unidades'"></span>
50                         <span th:unless="${p.existencias > 0}" class="badge text-bg-danger ms-2">Agotado</span>
51                     </div>
52                     <form th:action="@{/carrito/agregar}" th:method="POST" class="d-inline">
53                         <input type="hidden" th:value="${p.idProducto}" name="idProducto"/>
54                         <button type="submit"
55                             class="btn btn-sm btn-primary"
56                             th:disabled="${p.existencias == 0}"
57                             onclick="addCart(this.form)">
58                             <i class="fas fa-cart-plus"></i>
59                         </button>
60                     </form>
61                 </div>
62             </div>
63         </div>
64     </div>
65 </section>
66 </body>
67 </html>
```

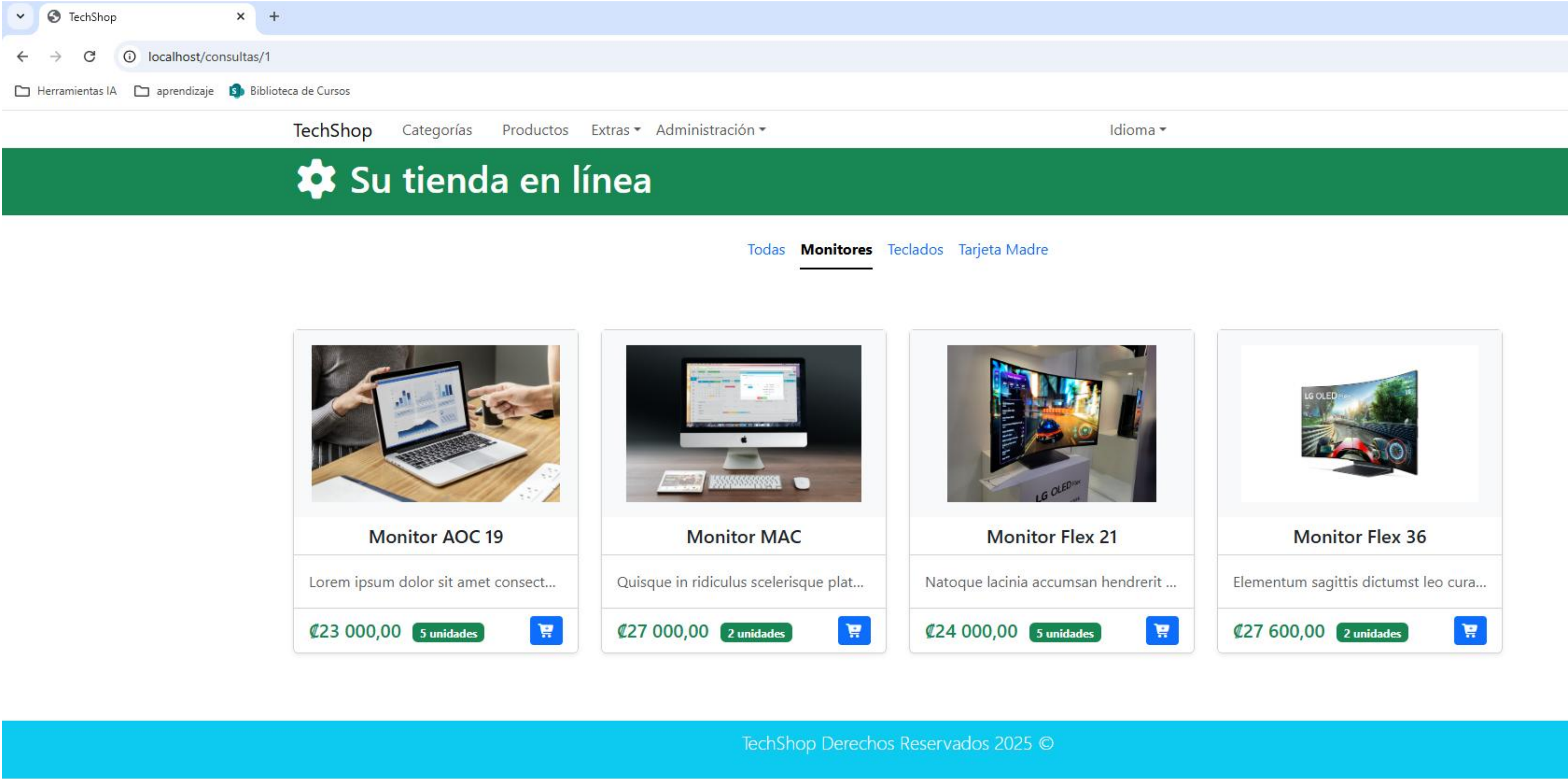
index.html

Se incluye el llamado al fragmento tarjetas en el index.html



```
<!-- index.html -->
<!DOCTYPE html>
<!-- Esta es la página principal del sitio web TechShop -->
<html>
  <head th:replace="~{general/fragmentos :: head}"/>
  <body>
    <header th:replace="~{general/fragmentos :: header}"/>
    <section th:replace="~{consultas/fragmentos :: tabs}"/>
    <section th:replace="~{consultas/fragmentos :: tarjetas}"/>
    <footer th:replace="~{general/fragmentos :: footer}"/>
  </body>
</html>
```

Cómo se ve ahora



Nos vemos en semana 7

RETO: ¿Cómo se puede hacer para que las tarjetas “crezcan” cuando el mouse pasar sobre una?

Tip: usa un poco de css, válidalo con Gemini...